

An Analysis of Multilayer perceptron

Multilayer perceptron

$$y = \text{sign}(w \cdot x + b)$$

$$w \leftarrow w + \eta(y - \hat{y})x$$

Multilayer perceptron -- Part 1

$$-\frac{\partial E(n)}{\partial v_j(n)} = e_j(n) \phi'(v_j(n)) \quad \frac{\partial E(n)}{\partial v_j(n)} = e_j(n) \phi'^{\prime}(v_j(n))$$

Multilayer perceptron -- Part 2

$y(v_i) = \tanh(v_i)$ and $y(v_i) = \frac{1}{1 + e^{-v_i}}$. The first is a hyperbolic tangent that ranges from -1 to 1 , while the other is the logistic function, which is similar in shape but ranges from 0 to 1 . Here y_i is the output of the i th node (neuron) and v_i is the weighted sum of the input connections. Alternative activation functions have been proposed, including the rectifier and softplus functions. More specialized activation functions include radial basis functions (used in radial basis networks, another class of supervised neural network models). In recent developments of deep learning the rectified linear unit (ReLU) is more frequently used as one of the possible ways to overcome the numerical problems related to the sigmoids.

Multilayer perceptron -- Part 3

where $y_i(n)$ is the output of the previous neuron i , and η is the learning rate, which is selected to ensure that the weights quickly converge to a response, without oscillations. In the previous expression, $\frac{\partial E(n)}{\partial v_j(n)}$ denotes the partial derivate of the error $E(n)$ according to the weighted sum $v_j(n)$ of the input connections of neuron i . The derivative to be calculated depends on the induced local field v_j , which itself varies. It is easy to prove that for an output node this derivative can be simplified to

Multilayer perceptron -- Part 4

$$-\frac{\partial E(n)}{\partial v_j(n)} = \phi'(v_j(n)) \sum_k -\frac{\partial E(n)}{\partial v_k(n)} w_{kj}(n)$$

$v_j(n) = \phi'(v_j(n)) \sum_k -\frac{\partial E(n)}{\partial v_k(n)} w_{kj}(n)$. This depends on the change in weights of the k th nodes, which represent the output layer. So to change the hidden layer weights, the output layer weights change according to the derivative of the activation function, and so this algorithm represents a backpropagation of the activation function.

Multilayer perceptron -- Part 5

Timeline In 1943, Warren McCulloch and Walter Pitts proposed the binary artificial neuron as a logical model of biological neural networks. In 1958, Frank Rosenblatt proposed the multilayered perceptron model, consisting of an input layer, a hidden layer with randomized weights that did not learn, and an output layer with learnable connections. In 1962, Rosenblatt published many variants and experiments on perceptrons in his book Principles of Neurodynamics, including up to 2 trainable layers by "back-propagating errors". However, it was not the backpropagation algorithm, and he did not have a general method for training multiple layers. In 1965, Alexey Grigorevich Ivakhnenko and Valentin Lapa published Group Method of Data Handling. It was one of the first deep learning methods, used to train an eight-layer neural net in 1971. In 1967, Shun'ichi Amari reported the first multilayered neural network trained by stochastic gradient descent, was able to classify non-linearly separable pattern classes. Amari's student Saito conducted the computer experiments, using a five-layered feedforward network with two learning layers. Backpropagation was independently developed multiple times in early 1970s. The earliest published instance was Seppo Linnainmaa's master thesis (1970). Paul Werbos developed it independently in 1971, but had difficulty publishing it until 1982. In 1986, David E. Rumelhart et al. popularized backpropagation. In 2003, interest in backpropagation networks returned due to the successes of deep learning being applied to language modelling by Yoshua Bengio with co-authors. In 2021, a very simple NN architecture combining two deep MLPs with skip connections and layer normalizations was designed and called MLP-Mixer; its realizations featuring 19 to 431 millions of parameters were shown to be comparable to vision transformers of similar size on ImageNet and similar image classification tasks.

Multilayer perceptron -- Part 6

In deep learning, a multilayer perceptron (MLP) is a kind of modern feedforward neural network consisting of fully connected neurons with nonlinear activation functions, organized in layers, notable for being able to distinguish data that is not linearly separable. Modern neural networks are trained using

An Analysis of Multilayer perceptron

Multilayer perceptron

backpropagation and are colloquially referred to as "vanilla" networks. MLPs grew out of an effort to improve on single-layer perceptrons, which could only be applied to linearly separable data. A perceptron traditionally used a Heaviside step function as its nonlinear activation function. However, the backpropagation algorithm requires that modern MLPs use continuous activation functions such as sigmoid or ReLU. Multilayer perceptrons form the basis of deep learning, and are applicable across a vast set of diverse domains.

Multilayer perceptron -- Part 7

The MLP consists of three or more layers (an input and an output layer with one or more hidden layers) of nonlinearly-activating nodes. Since MLPs are fully connected, each node in one layer connects with a certain weight w_{ij} to every node in the following layer.

Multilayer perceptron -- Part 8

$E(n) = \frac{1}{2} \sum_j e_j^2(n)$ $\{\text{output node } j\}$. Using gradient descent, the change in each weight w_{ij} is

Multilayer perceptron -- Part 9

Learning occurs in the perceptron by changing connection weights after each piece of data is processed, based on the amount of error in the output compared to the expected result. This is an example of supervised learning, and is carried out through backpropagation, a generalization of the least mean squares algorithm in the linear perceptron. We can represent the degree of error in an output node j in the n th data point (training example) by $e_j(n) = d_j(n) - y_j(n)$, where $d_j(n)$ is the desired target value for n th data point at node j , and $y_j(n)$ is the value produced by the perceptron at node j when the n th data point is given as an input. The node weights can then be adjusted based on corrections that minimize the error in the entire output for the n th data point, given by